CS 3251 - Computer Networks I

Fall 2026

Programming Assignment

DUE DATE: November 2, 11:59PM, submit on Canvas

Collaboration, AI or Human Assistance ***is Allowed*** with **Appropriate Citation**

Completion Grade – All steps must Be Attempted. Grade proportional to number of items attempted and not based on correctness of submission.

Introduction

In this assignment you will look at sockets programs and their properties using both TCP and UDP. The main objective is to get you to examine sockets code and be able to run it and work with it. The four program files referenced in the assignment are included in the attached .zip file.

Part 1 - Bulk Data Transfer Using TCP

The programs in **clientTCPwithbug.py** and **serverTCPwithbug.py** deploy an **in-memory TCP throughput benchmarker** designed to measure network performance without disk I/O bottlenecks.

- **Server:** Runs continuously on the loopback interface (127.0.0.1), listening for client requests. When a client requests a specific payload size, the server streams a generated, repeating alphabet pattern (80 'a's through 80 'z's) entirely from RAM.

- **Client:** Sends a CLI-specified payload size request to the server, measures the transfer duration, extracts and displays the first 320 characters to verify data integrity, discards the remaining incoming bytes, and outputs the resulting throughput in **Mbps**.

Follow these steps for the assignment.

Step 1: Fix bugs:

Submit the corrected code and use it in subsequent steps.

- In server there is a missing “bind” statement. Insert it in the right place with the right parameters.
- In the client there is a missing “connect” statement. Insert it in the right place with the right parameters.

Important Note: If you are stuck with this step please check with a TA or one of the instructors and we will give you the correct code. The rest of this part assumes you have working code with no bugs.

Step 2: Experiment with the system

There is nothing to submit for this step

Experiment with the client downloading a certain number of bytes. Here are some more details of what the code does:

Application Protocol Specification

The application uses a lightweight, custom text-to-binary protocol over TCP.

Client Request:

Since there are no physical files, the client dictates how much data it wants to "download". The client initiates the transfer by sending a single text string specifying the requested payload size in bytes, terminated by a newline character (\n).

- **Format:** SIZE:<number_of_bytes>\n
- **Example:** SIZE:1048576\n (Requests 1 MB of data)

Server Response:

The server does not send any headers back. As soon as it parses the requested size, it immediately streams raw binary data matching the specified sequence (80 'a's, 80 'b's, ..., 80 'z's, repeating). The server stops sending and closes the connection once the exact number of requested bytes has been transmitted.

How to Run

1. **Start the Server:**

Open a terminal and run the server script. It takes no arguments and will simply start listening on the hardcoded loopback port.

python serverTCPwithbug.py
Expected Output: Server listening on 127.0.0.1:65432...

2. **Run the Client:**

Open a **second** terminal. Run the client script and pass the desired file size in bytes as the only positional argument.

Example: Requesting a 10 MB (10,485,760 bytes) file:

python clientTCPwithbug.py 10485760

3. **Interpreting the Output:**

- The server terminal will log the incoming connection, state how many bytes it sent, and immediately return to a listening state.
- The client terminal will print the exact first 320 characters it received (you will see 80 “a”s, 80 “b”s, 80 “c”s, and 80 ”d”s).This is to confirm that at least the beginning of the data is received reliably and in sequence (as is expected with TCP).
- Below the characters, the client will display the **timing** (measured strictly from the perf_counter timestamp taken directly prior to dispatching the request to the timestamp directly after the final chunk is received) and the **throughput** in Megabits per second (Mbps).

Important Note: The client will exit after it completes its file download. While the server will stay waiting for another client request. You can exit the server with a CTRL C and then running a client one more time. Both will then exit with errors. **Bonus optional question: Can you explain why this is happening?** This is not ideal behavior. It can be fixed with timeouts at the server. But we chose not to add this code to simplify things.

Step 3: Answer the following questions:

Submit answers to these questions:

Question 1: The TCP Stream

In the client code, data is received using a while bytes_received < file_size: loop, and recv() is called repeatedly. Why can't the client simply call client_socket.recv(file_size) exactly once and expect to receive the entire requested file?

Question 2: Server Socket Roles

In the server code, two distinct socket objects are used: server_socket and conn (which is returned by accept()). Explain the specific role of each of these sockets. Why does the server need a new socket for the connection instead of just communicating over server_socket?

Question 3: Sending Data

Both the client and the server use the sendall() method instead of the standard send() method. From a socket programming perspective, what specific problem does sendall() solve that send() does not guarantee?

Question 4: Socket Options

The server code includes the following line before binding the socket:

server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

If you were to remove this line, stop the server while a transfer is happening, and immediately try to restart the server, you would likely get an OSError: [Errno 98] Address already in use. Why does the operating system hold onto the port, and how does this line fix it?

Question 5: Application-Layer Framing

The application protocol requires the client to terminate its request with a newline character (e.g., SIZE:1048576\n). If we removed this newline requirement and the client simply sent SIZE:1048576, what fundamental problem would the server face when trying to read the request over a TCP socket?

Step 4: TCP Performance Measurement and Throughput Analysis

Submit the graph and observations requested. We don't need to see the output from your runs.

Objective:

Measure how the requested file size impacts the actual throughput of your socket application. Because network performance is influenced by random operating system events (like CPU context switching, memory allocation, and background processes), you will run multiple trials to calculate a reliable average and smooth out these random effects.

Instructions:

1. **Data Collection:**

Run your client script to request file transfers of varying sizes. You must measure at least **six different file sizes**, spanning from 1,024 bytes (1 KB) up to 100 Megabytes (104,857,600 bytes).

- *Recommended sizes:* 1 KB, 10 KB, 100 KB, 1 MB, 10 MB, and 100 MB.
- For *each* file size, run the client **three separate times**.
- Record the throughput (in **Mbps**) reported by the client for each run, and calculate the **average throughput** of the three runs for that specific file size.
- Note: There is some random OS and other effects that will affect the throughput measurement. Taking the average of multiple measurements can smooth out these effects and give you more statistically valid measurements.

2. **Graphing the Results:**

Create a line graph or scatter plot with connecting lines using a tool of your choice (Excel, Python/Matplotlib, Google Sheets).

- **X-Axis:** Requested File Size in bytes. *(Hint: Because the file sizes span several orders of magnitude, use a **logarithmic scale** for the X-axis so all points are clearly spaced).*
- **Y-Axis:** Average Throughput in Megabits per second (**Mbps**).

- Ensure your axes are clearly labeled and the graph includes a descriptive title.

3. **Analysis and Observations:**

Below your graph, write a short analysis providing **2 to 3 distinct observations** about the data. Address the following in your analysis:

- What happens to the throughput (in Mbps) when transferring very small files (e.g., 1 KB) compared to larger files? Based on how TCP connections are established and the script's timing placement, why is the measured throughput so low for tiny transfers?
- Does the throughput in Mbps increase indefinitely as file size grows, or does it eventually plateau/level off as you approach 100 MB? Since you are running this on the loopback interface (127.0.0.1) without physical network cables, what system resources limit the maximum achievable throughput?

Part 2 - Bulk Data Transfer Using UDP

Next you will work the UDP client and server in **UDPclientARQLoss.py** and **UDQserverARQLoss.py.**

This is the same application as in Part 1 except we are now using UDP. But because UDP is connectionless and can lose data the application deploys its own stop and wait ARQ protocol (discussed in class) with the server as the sender and the client as the receiver (no ARQ is deployed in the other direction). Please note the relative complexity of this code when compared with the TCP code. The main difference is that this code needs to take care of reliability and sequencing whereas the TCP code did not. This is the main reason why TCP is preferred in many applications.

Step1: Experiment with the system

There is nothing to submit for this Step.

There are no known bugs in the code provided. So you can test out without any additional code. Here are some more details:

Application Protocol Specification
The client and server communicate over UDP using a simple request-response protocol designed for a lossless environment:

1. **Connection Setup**: None (UDP is connectionless).
2. **Request Phase**: The client sends a single UDP datagram to the server containing the requested file size as a UTF-8 encoded integer string (e.g., "1048576" for a 1 MB file). The client request will also include the timeout period for the ARQ protocol in seconds, the data loss probability and the ACK loss probability.

3. **Data Transfer Phase**:

    - Upon receiving the request, the server parses the file size and begins transmitting the file data in chunks back to the client's address and port.
    - The data consists of a repeating pattern: 80 'a's, 80 'b's, ..., 80 'z's (a 2080-byte repeating sequence).
    - To maintain the pattern seamlessly while maximizing UDP efficiency, the server uses a chunk size of 8320 bytes (exactly 4 complete cycles of the 2080-byte pattern). The final chunk is sliced to match the exact remaining bytes requested.
4. **Termination**: Because UDP lacks a built-in EOF marker, termination is implicitly handled by the client. The client continuously adds the length of received datagrams to a counter and exits the receive loop once the received bytes equal the requested file size.

ARQ Protocol Details

To implement Stop-and-Wait ARQ above UDP, we introduce a custom 4-byte header for every packet:

- **Byte 0 (Type):** 0 = Request (REQ), 1 = Request Acknowledgment (RAK), 2 = Data (DAT), 3 = Data Acknowledgment (ACK).
- **Byte 1 (Sequence):** Alternating 0 or 1 for the Stop-and-Wait sequence number.
- **Bytes 2-3 (Payload Length):** A 16-bit unsigned integer representing the length of the payload following the header.

Protocol Flow:

1. **Request Phase:** The client sends a REQ packet containing the file size and the user-defined timeout value. The client starts a timer. If it does not receive a RAK packet from the server before the timeout, it retransmits the REQ.
2. **Data Phase:** The server configures its socket with the client's requested timeout. It sends a DAT packet with sequence 0 and waits for an ACK for sequence 0.
    - If the server receives the correct ACK, it flips the sequence number to 1, sends the next chunk, and waits for ACK 1.
    - If the server times out before receiving the ACK (indicating packet loss of either the DAT or the ACK), it retransmits the current DAT packet.
3. **Client Reception:** The client waits for DAT packets. If it receives a packet with the expected sequence number, it processes the data, flips its expected sequence number, and sends an ACK. If it receives a duplicate packet (meaning its previous ACK was lost), it ignores the data but *resends the ACK* so the server can move forward.

Protocol Operation with Simulated Loss

The Stop-and-Wait ARQ protocol inherently handles lost packets by using timeouts and sequence numbers.

1. **Data Loss:** When the client receives a Data (DAT) packet, it rolls a random number. If it falls below data_loss_prob, the client simply ignores the packet and does not send an Acknowledgment (ACK). The server's timer will expire, causing it to retransmit the identical DAT packet.

2. **ACK Loss:** The client communicates the user-defined ack_loss_prob to the server in the initial REQ packet. When the server receives an ACK, it rolls a random number. If it falls below ack_loss_prob, the server ignores the ACK.
3. **Duplicate Detection:** Because the server ignored the ACK, its timer will expire, and it will retransmit the *same* DAT packet. The client will receive this DAT packet, but its sequence number will not match the client's expected_seq. The client identifies this as a duplicate, drops the payload (preventing corruption), but **resends the ACK** for that sequence number so the server can finally move forward.
4. **Validation:** The client now captures 1,040 characters. Despite packets being lost and duplicated, the printed output will remain perfectly sequential because the ARQ protocol shields the application layer from network anomalies.

How to Run:

Run the server first as usual:

```
python UDPserverARQLoss.py
```

Now execute the client, providing the 4 required arguments: [size] [timeout] [data_loss_prob] [ack_loss_prob].

For example, to request a 10 MB file with a 0.5s timeout, a **30% chance of dropping data**, and a **10% chance of dropping ACKs**:

```
python UDPclientARQLoss.py 10485760 0.5 0.3 0.1
```

Step 2: Answer the following questions

Submit answers to these questions

Question 1: Server Binding vs. Client Socket Setup

In server.py, the code includes server_socket.bind(('127.0.0.1', 12000)), but client.py never calls bind().

1. Why is calling bind() mandatory for the server socket?
2. What happens to the client socket's port number when the client executes its first sendto() call without having called bind()?

Question 2: Transmitting Data with sendto()

In UDP socket programming, we use sendto() rather than send().

1. What two arguments does sendto() require?
2. Why does UDP require you to pass the destination address to every send call, whereas standard TCP send() calls do not?

Question 3: Receiving Data with recvfrom()

When the server receives a request from a client, it executes:

"packet, client_address = server_socket.recvfrom(65535)"

1. What two pieces of information are returned in the tuple by recvfrom()?
2. What does the parameter 65535 represent?
3. Why is the client_address returned by recvfrom() essential for the server when replying to a UDP client?

Step 3: UDP Performance Measurement and Throughput Analysis

Submit the graph and observations requested. We don't need to see the output from your runs.

Using this UDP application, follow the same instructions for Step 4 of Part 1. The difference is now we have three additional parameters: Timeout period, Data Loss Probability and ACK Loas Probability.

1. Draw four graphs of throughput as a function of bulk transfer size with the following parameter settings:
    a) Timeout = 0.5 seconds, ACK and Data Loss = 0.0
    b) Timeout = 0.5 seconds, ACK and Data Loss =0.01
    c) Timeout =1.0 seconds ACK and Data Loss =0.01
    d) Timeout =0.5 seconds ACK and Data Loss = 0.1
2. Analysis and Observation:

- Below your graphs compare these graphs with each other and compare with the TCP graph you obtained in Part1. When comparing with TCP it only make sense to compare your graph in a) above since the TCP experiments did not have any packet losses.
- When making this comparison do you see any advantage here relative to using TCP in terms of throughput performance? Could this justify the complexity of the UDP code? There is no correct answer here as it depends on the environment you run this. Just answer the question based on the data you obtained.